HMD OVERVIEW

SONY COMPUTER ENTERTAINMENT ™

A general introduction to HMD

Presenters:

Mark Breugelmans

James Russell

PlayStation ®

HMD Topics

- What is HMD?
- What can it do?
- Demos
- Features
- Basic Architecture
- Tools
- How fast is it?

What is HMD?

- HMD - Hierarchical Modelling Data
- The standard graphics format after TMD
  - Available now in current libraries
- Actively Developed by SCEI

What can it do? (demos)

- Hierarchical models
- Shared Polygons

Coord System1	Shared Polygon	Coord System2

Key frame animation, motion switching
Equipment - cameras and lights
Reflection mapping
More to come (for example shadows)

Features

- General features of the HMD format

Integrated file format

Before:

- TMD
- TOD
- TIM
- Other Data files

Integrated into HMD:

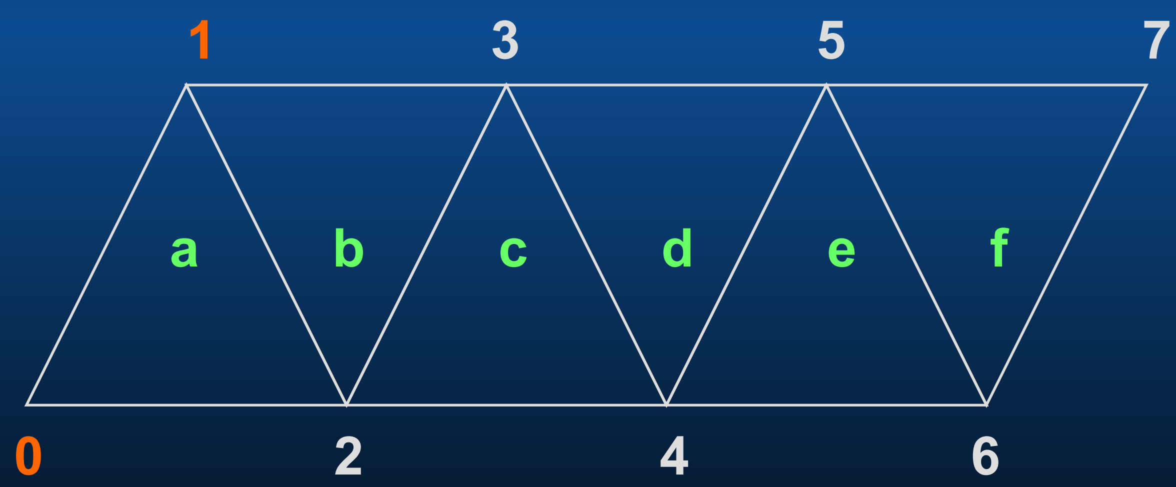
- Co-ordinate system
- Modelling data
- Images
- Animation
- Cameras and Lights
- Additional primitives

Programming interface

- Loosely linked to LIBGS libraries (4.0)
- Plug in functions to link what you need
  - Add you own functionality
- More control at file level (authoring)
  - Example: Animation sequences

High performance

- Key routines coded in assembly. Arguments passed using scratchpad
- Meshes reduce calculations and data size



- Or you can write your own drivers

Basic Architecture

- HMD Primitives
  - Non drawing primitives (processing except generation of poly drawing instruction)
  - Drawing primitives (Poly drawing instruction generation)
  - Each primitive uses its own driver in the code
- Data
  - Everything defined with relative addresses

Basic Architecture - part 2

- Primitive Sets (List of primitives)
  - Primitive type ID - To identify driver
  - Pointer to primitive header
  - Primitive data
- Mapping, Scanning and Drawing
  - GsScanUnit replaces TypeID with pointer to Driver
  - GsMapUnit transforms relative addresses
  - GsSortUnit allocates object to the ordering table

Tools

- Converters from RSD
- Tool vendors support (Release/Coming soon)
  - Lightwave
  - 3d studio max
  - Softimage
  - PowerAnimator
- HMD assembler / disassembler

Extensibility

- Plug in driver architecture
  - provides easy extension to the format
  - Add your own features / improve existing
  - Possible extensions from 3rd parties
- Define unique data structures
  - add application specific data: eg. bounding boxes
- Demo of added primitive driver

How fast is it

- Venus with 1d Environment mapping - 58,500 polygons per second
- Venus with 2d Environment mapping - 48,000 polygons per second
- Dinosaur with 1d Environment mapping and animation - 55,000 polygons per second

Summary

- Integrated file format
- Many extra features
- High performance
- More authoring power
- Extendable

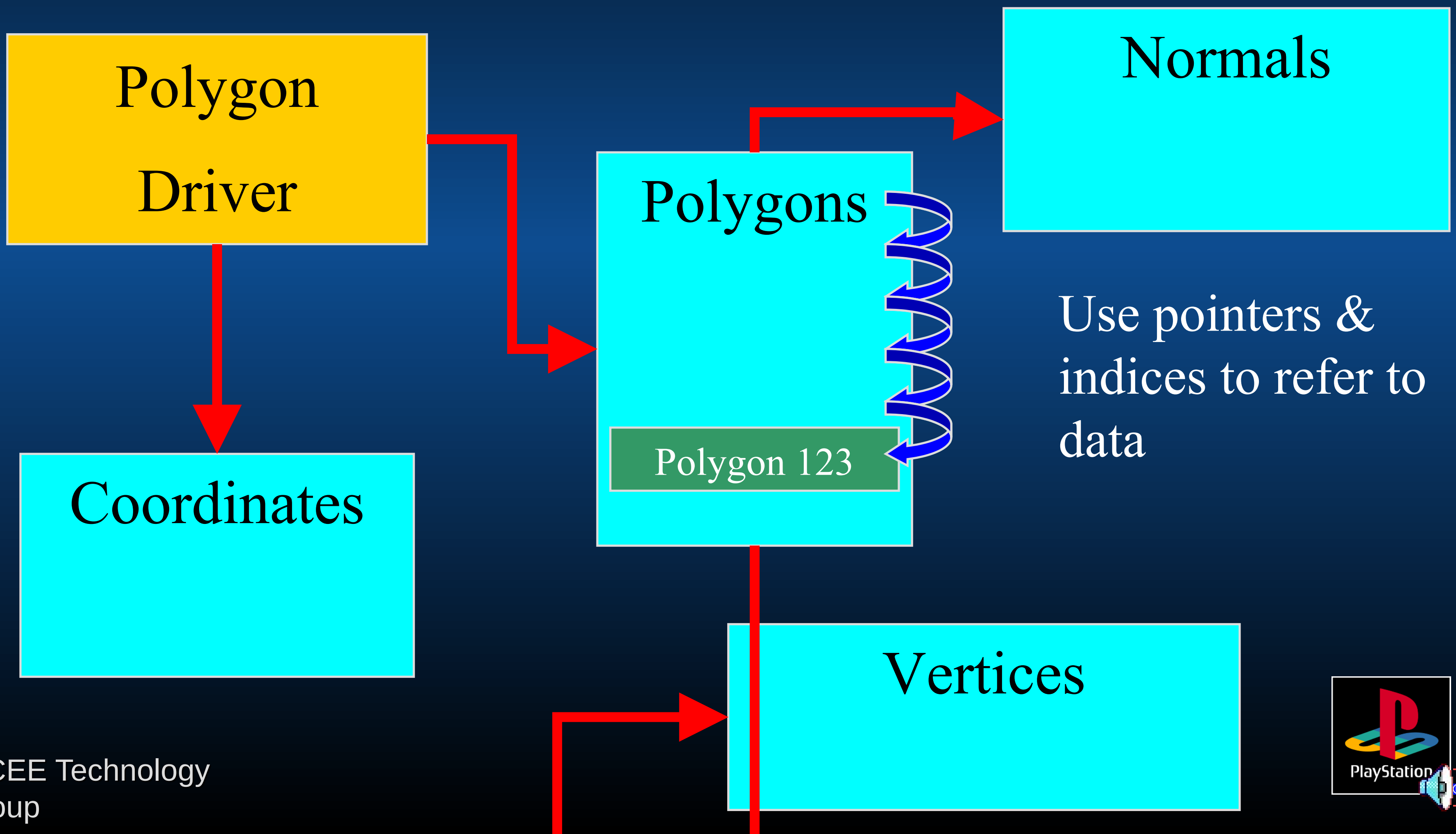
HMD internals

How does HMD work?

HMD philosophy

- Raw data is chunked into large sections
- “Primitives” bind parts of that raw data together
- Divide & conquer with many small & simple drivers

Raw data is chunked into large sections



Use pointers & indices to refer to data

An HMD file contains Primitive Blocks & raw data

HMD file

Primitive Block
Left Arm

Primitive Block
Right Arm

Primitive Block
Torso

Primitive Block
Head

3D Model Data

Animation Data

Texture Data

Primitive Blocks contain sets of Primitives

Primitive Block

Primitive

Triangles for pincers

Primitive

Triangles for shoulder

A Primitive Block is an object "part", and has its own coordinate system.

Primitive

Quadrangles for pincers

Primitive

Triangles for arm

Primitives provide a context for raw data

Primitive

Polygon index 0
Polygon index 1
Polygon index 2
...

Primitives...

- ...are the units processed by drivers.

Primitives provide a context for raw data

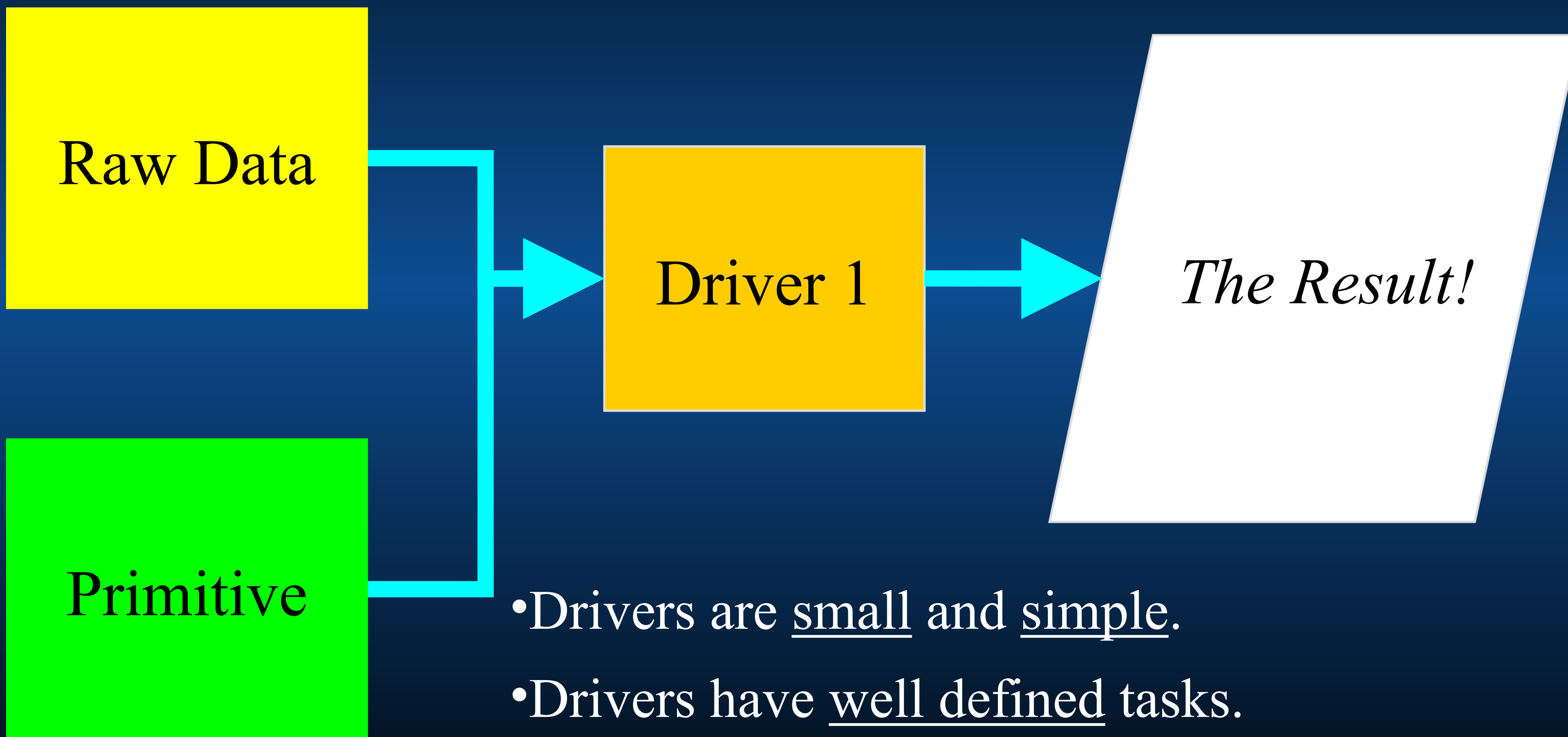
“Gouraud Triangles”

Polygon index 0
Polygon index 1
Polygon index 2
...

Primitives...

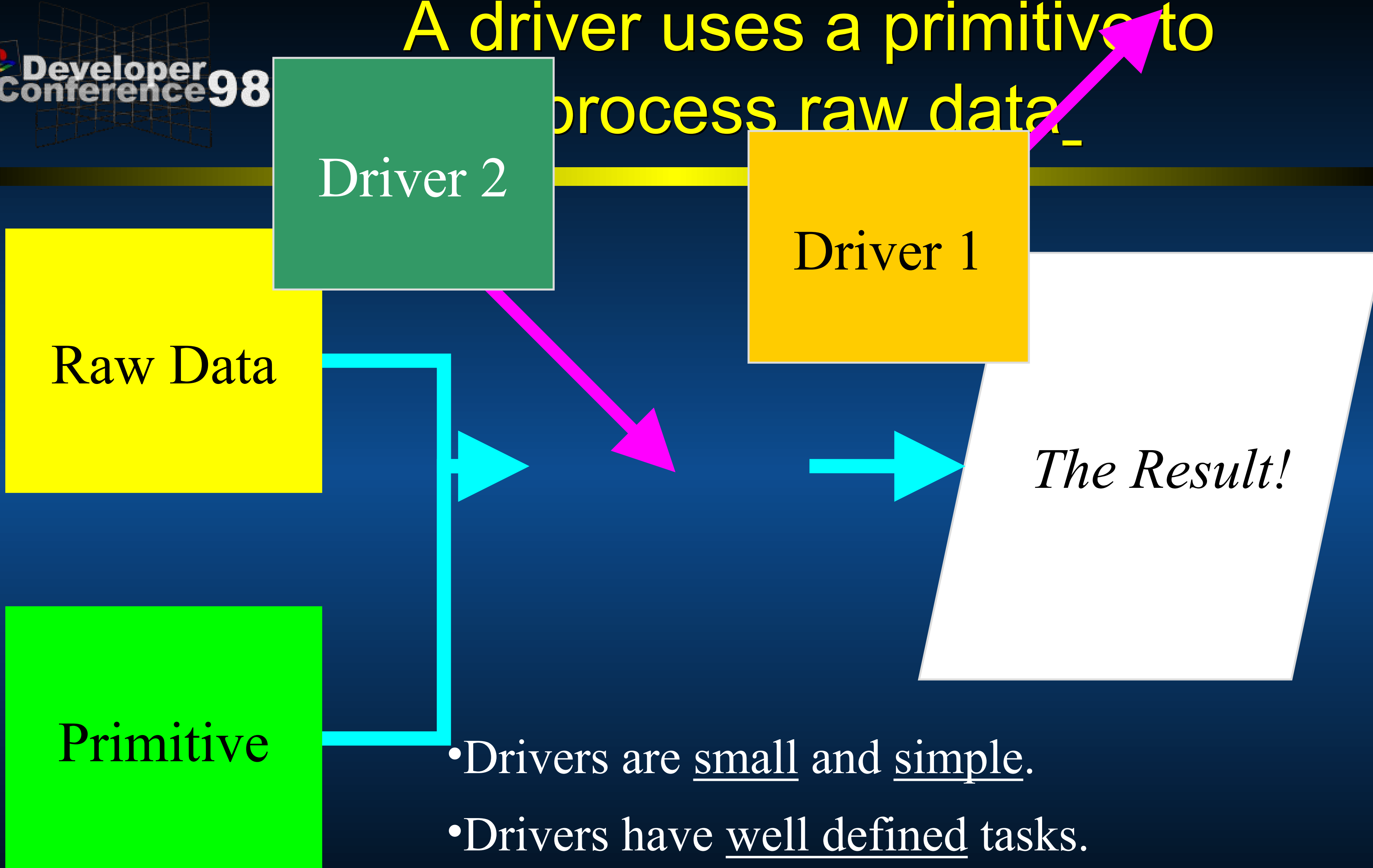
- ...are the <u>units</u> processed by drivers.
- …specify the type of driver they’ll need.

A driver uses a primitive to process raw data_



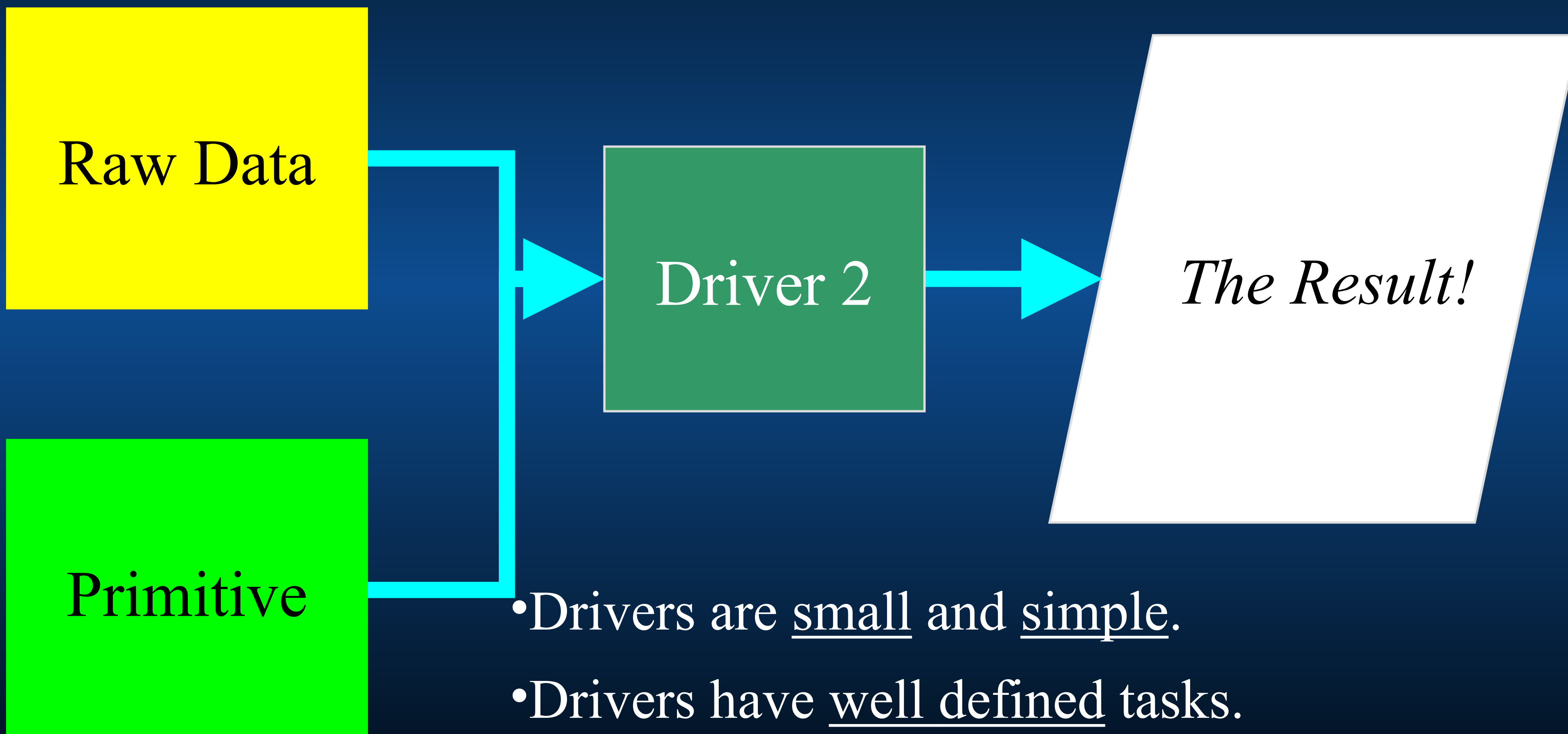
- Drivers are small and simple.
- Drivers have well defined tasks.
- Swap and extend drivers to do what *you* want.

A driver uses a primitive to process raw data



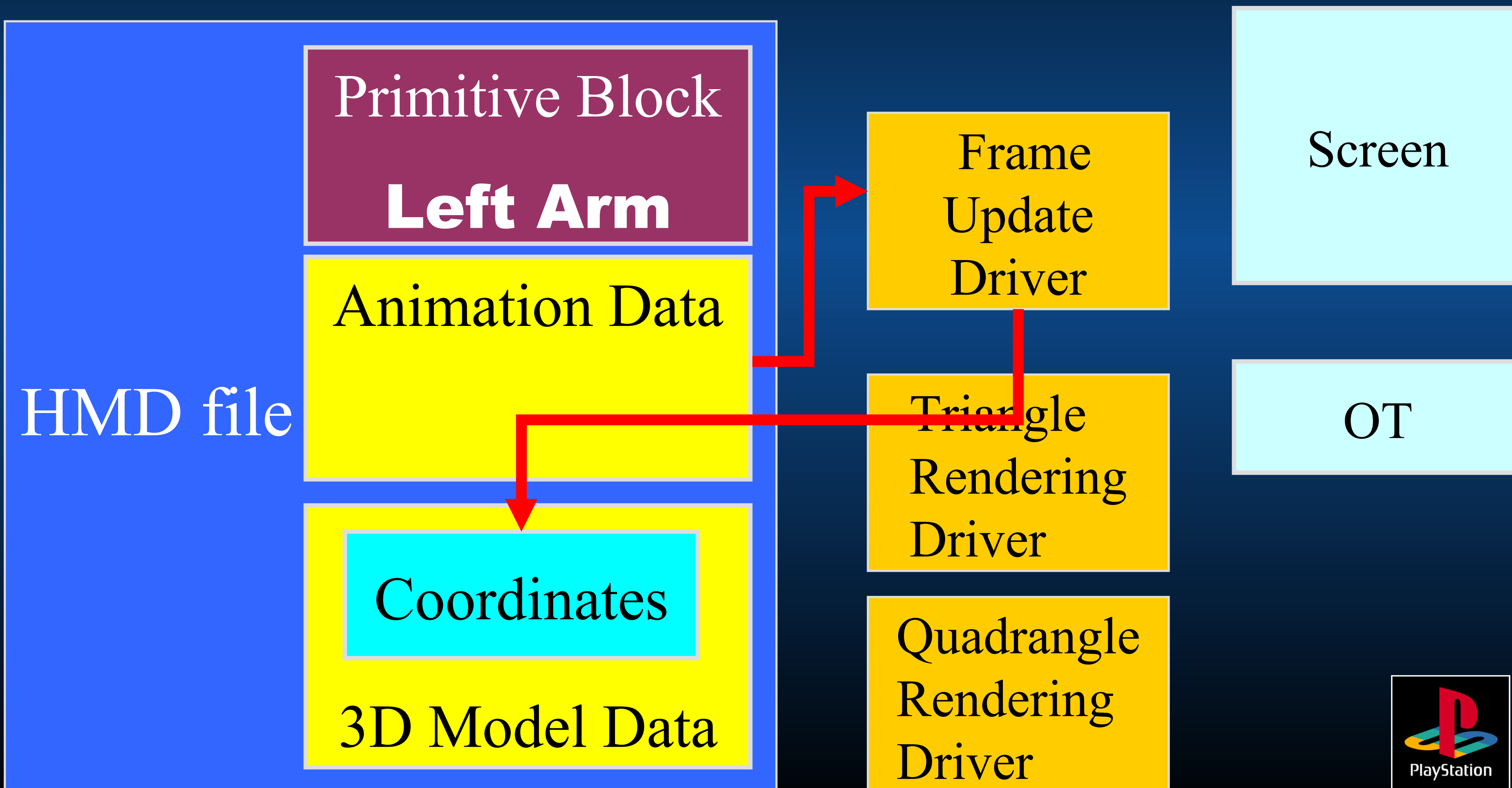
- Drivers are <u>small</u> and <u>simple</u>.
- Drivers have <u>well defined</u> tasks.
- Swap and extend drivers to do what *<u>you</u>* want.

A driver uses a primitive to process raw data_

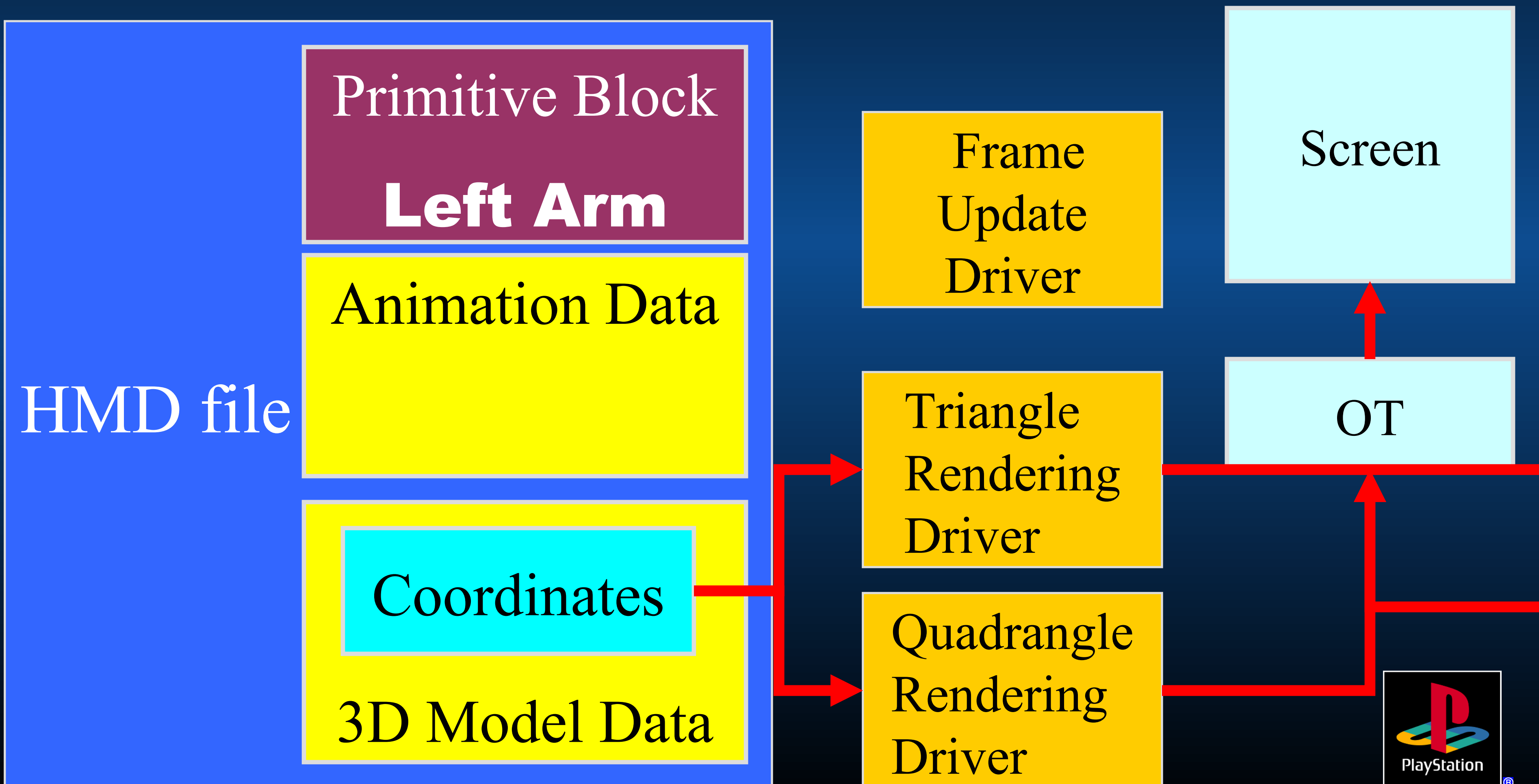


- Drivers are small and simple.
- Drivers have well defined tasks.
- Swap and extend drivers to do what *you* want.

The application uses many Drivers to get the final result



The application uses many Drivers to get the final result



GsSortUnit() processes Primitive Blocks

After initialisation, GsSortUnit() is the

ONLY

function you need to call to see your object on screen!

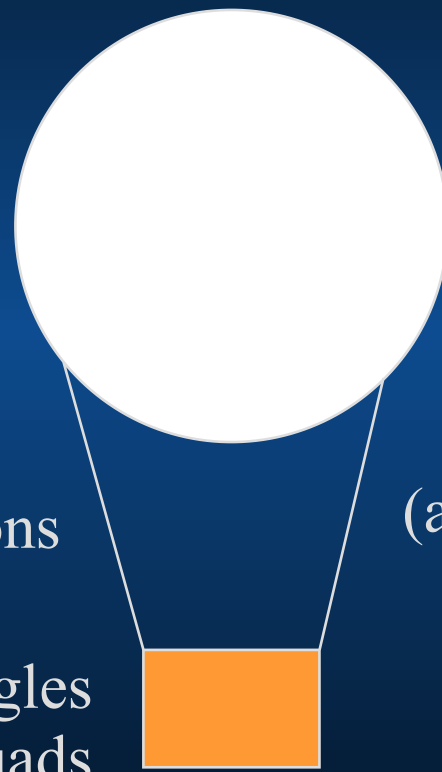
- Animation
- Polygons
- Shared Polygons
- MIMe

It's all handled by the Drivers!

Balloon.hmd example (1)

Types of primitives

Gouraud Triangles
Gouraud Quads



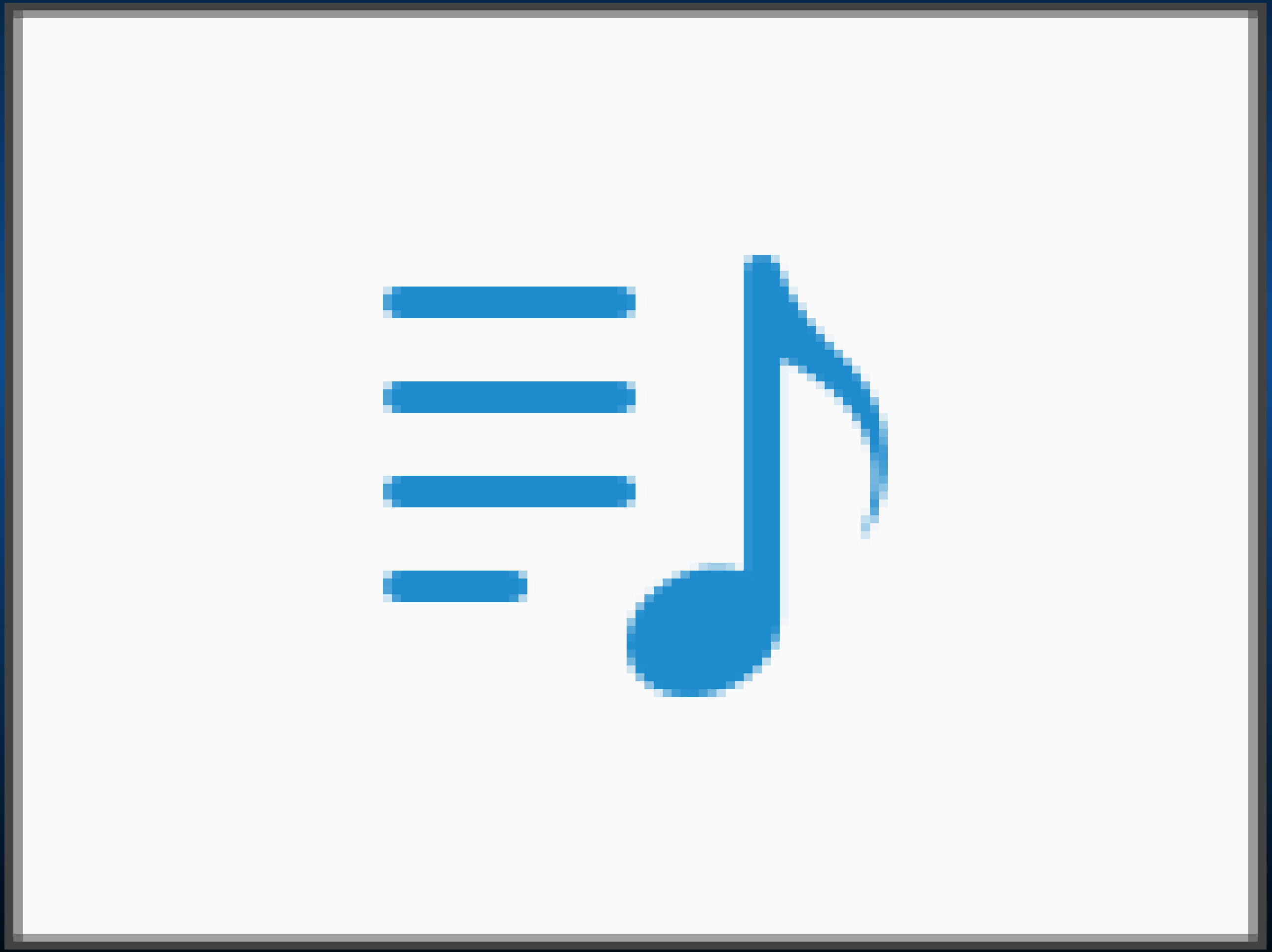
Shared Polygons

(and Animation Frames)

Gouraud Triangles
Gouraud Quads

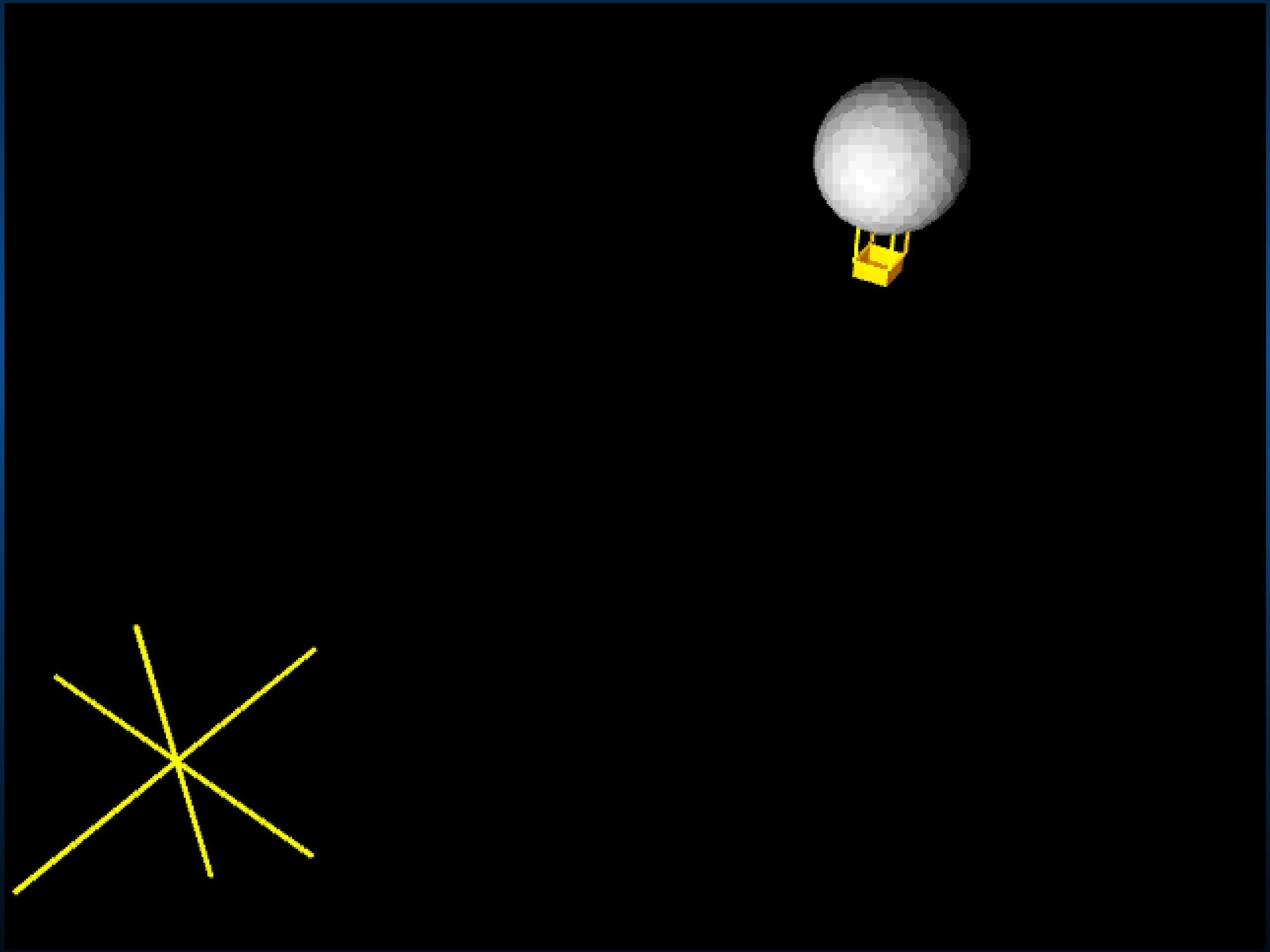
Balloon.hmd example (2)

Coordinate Systems



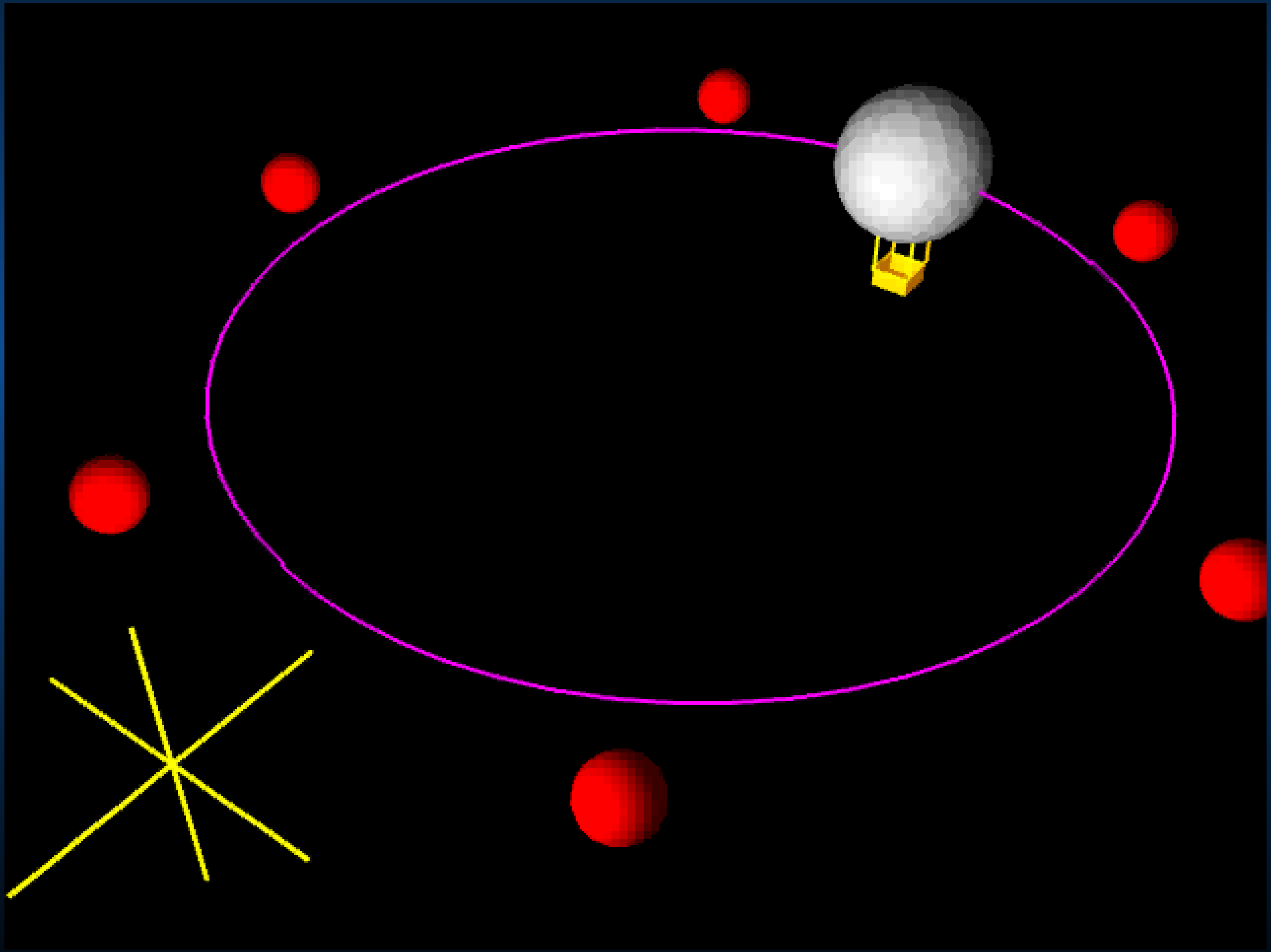
Balloon.hmd example (2)

Coordinate Systems



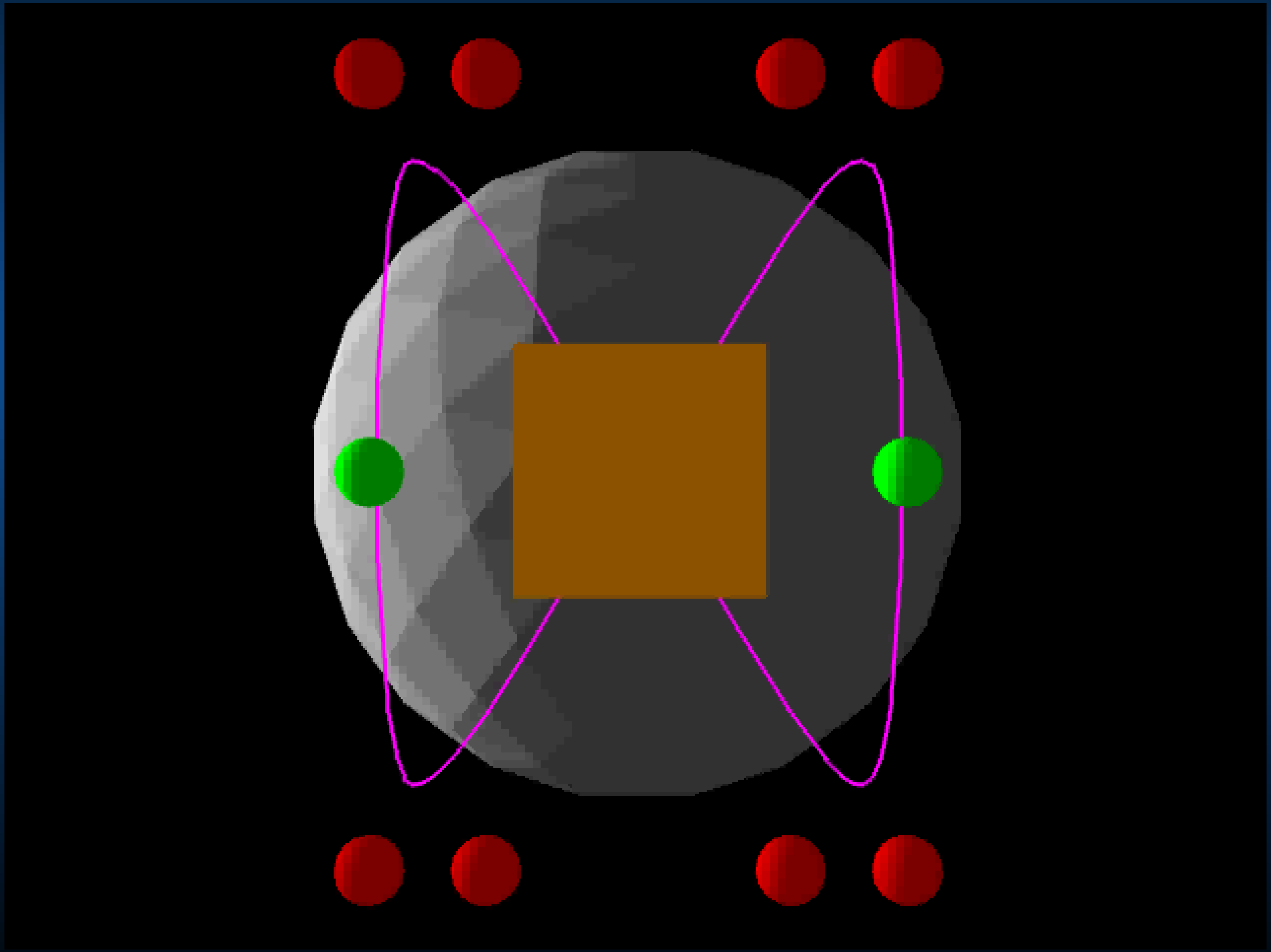
Balloon.hmd example (2)

Coordinate Systems



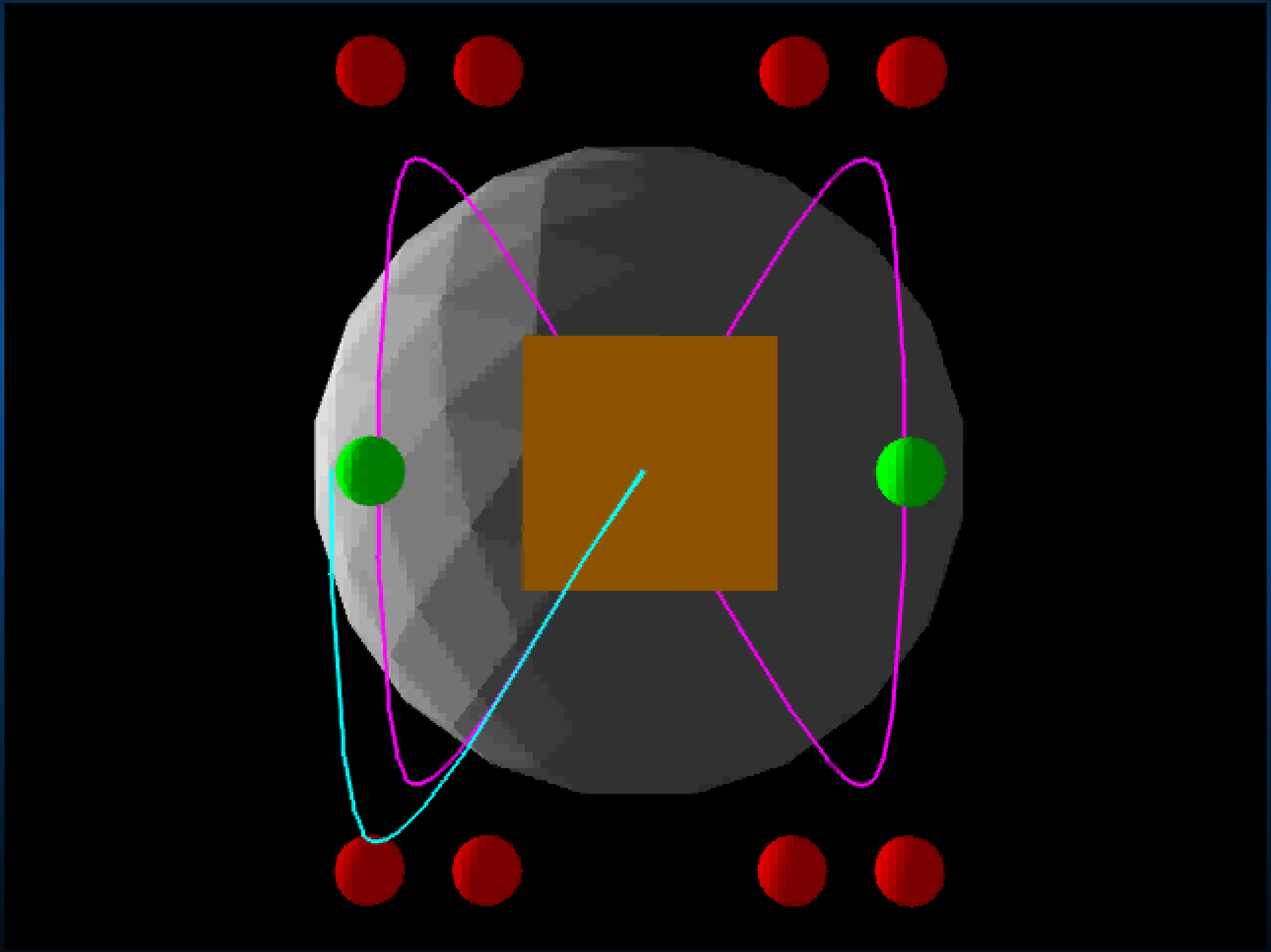
Balloon.hmd example (2)

Coordinate Systems



Balloon.hmd example (2)

Coordinate Systems



Balloon.hmd example (5)

Data required for the model

- Polygons
  - Normals
  - Vertices
  - Polygon connection information
  - Shared polygon connection information
  - Coordinate systems
- Animation
  - Bezier and B-Spline parameters
  - Sequence information

Putting it all together

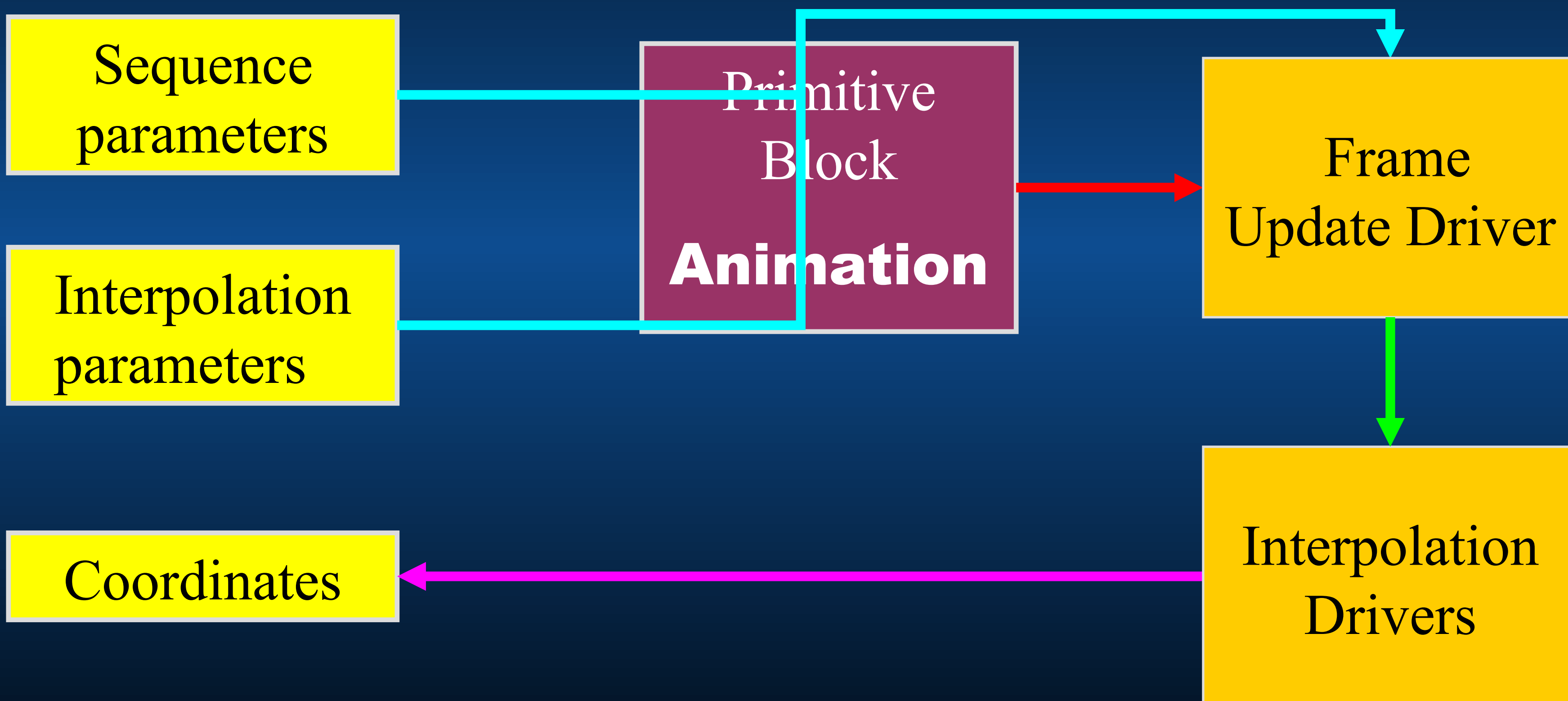
Primitive Blocks order

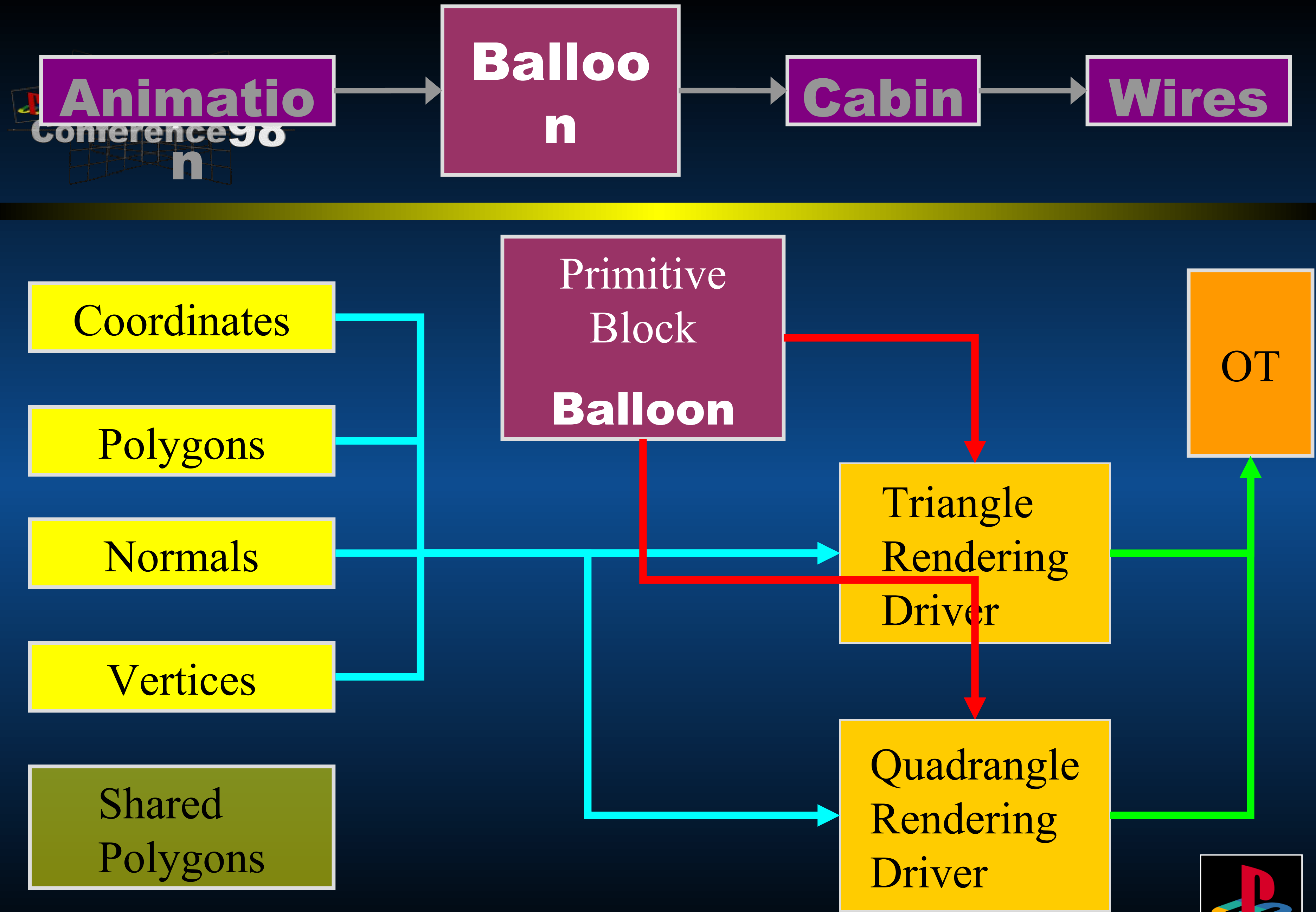
Animatio
n

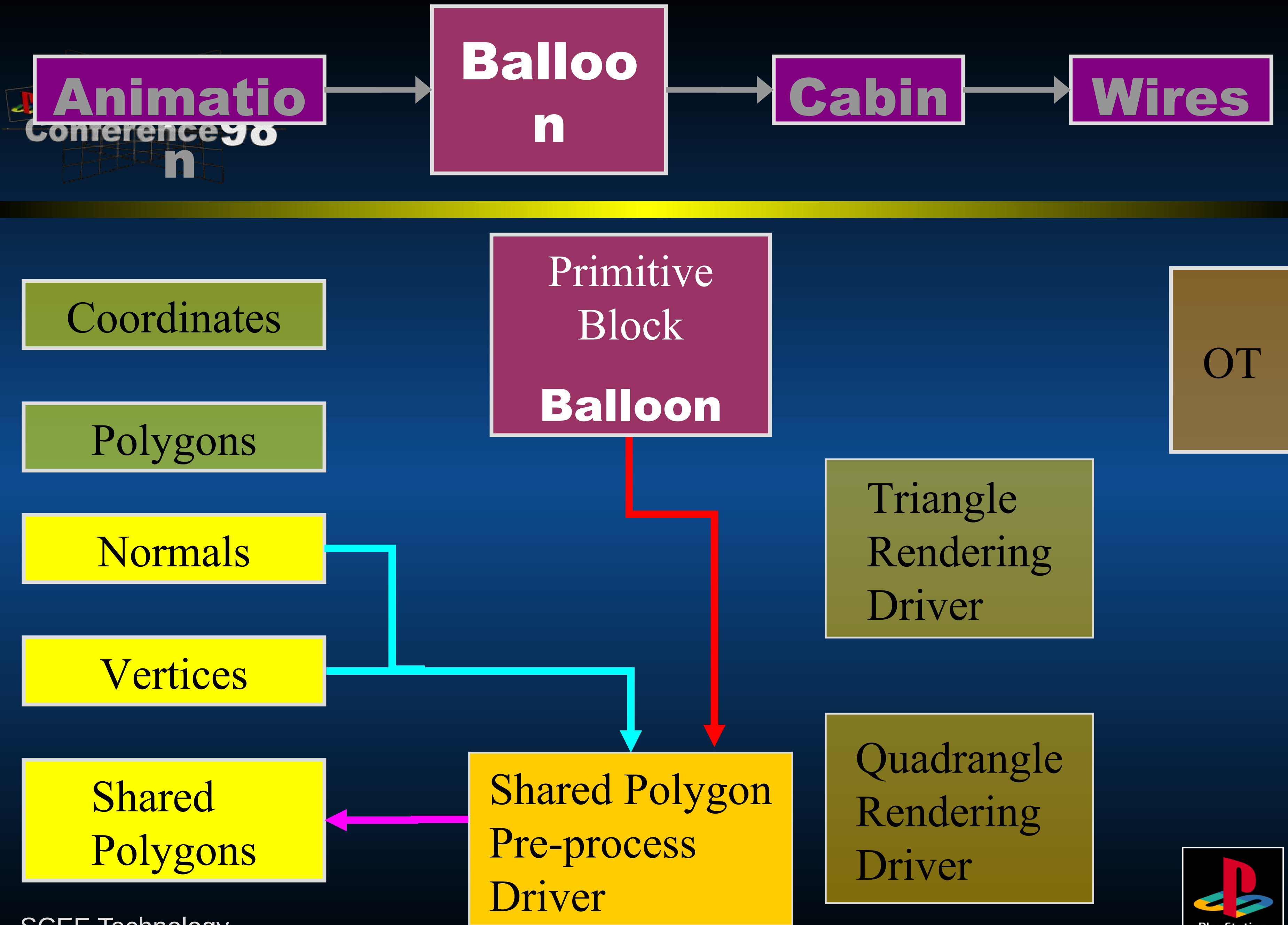
Balloon

Cabin

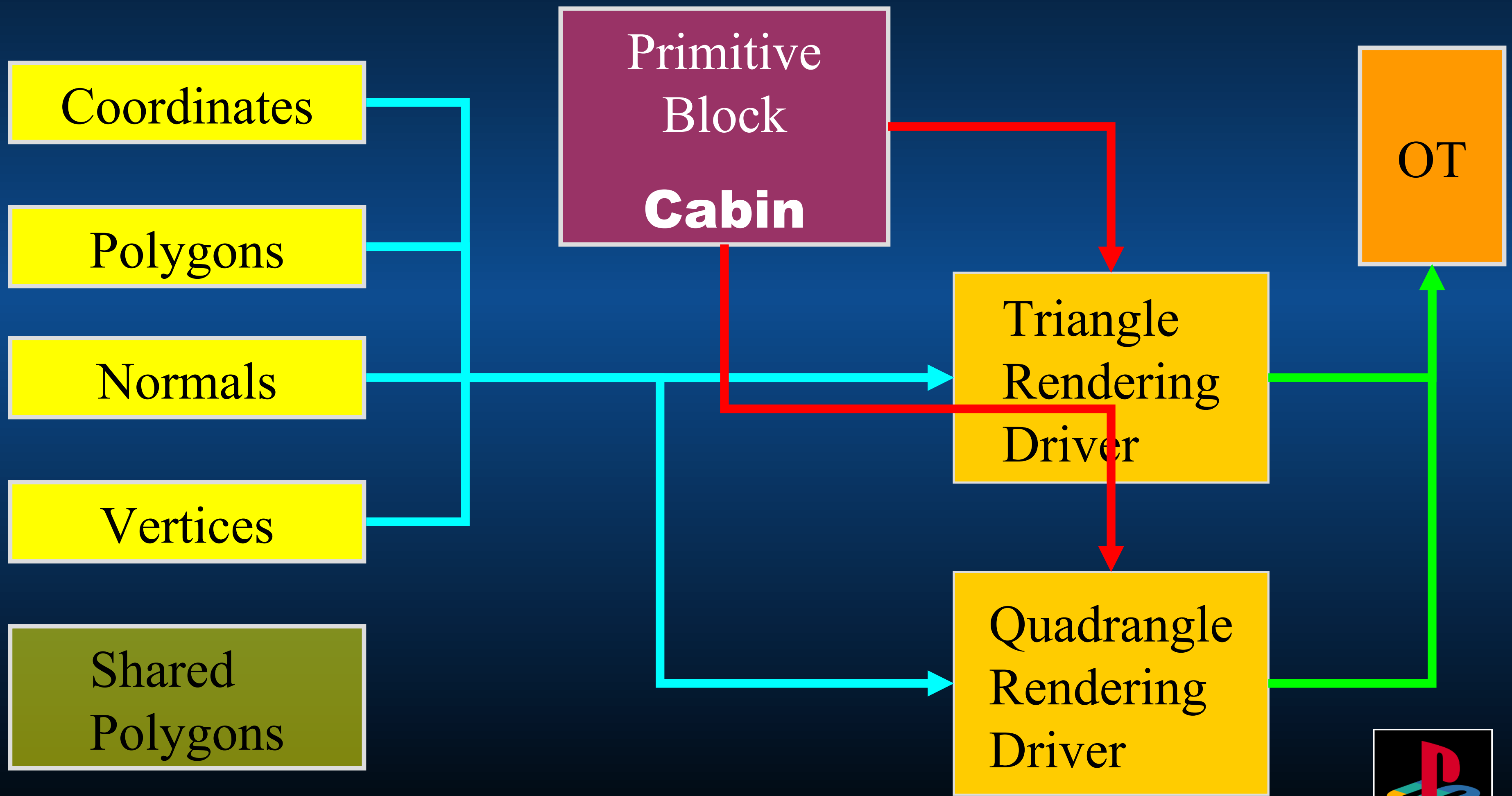
Wires

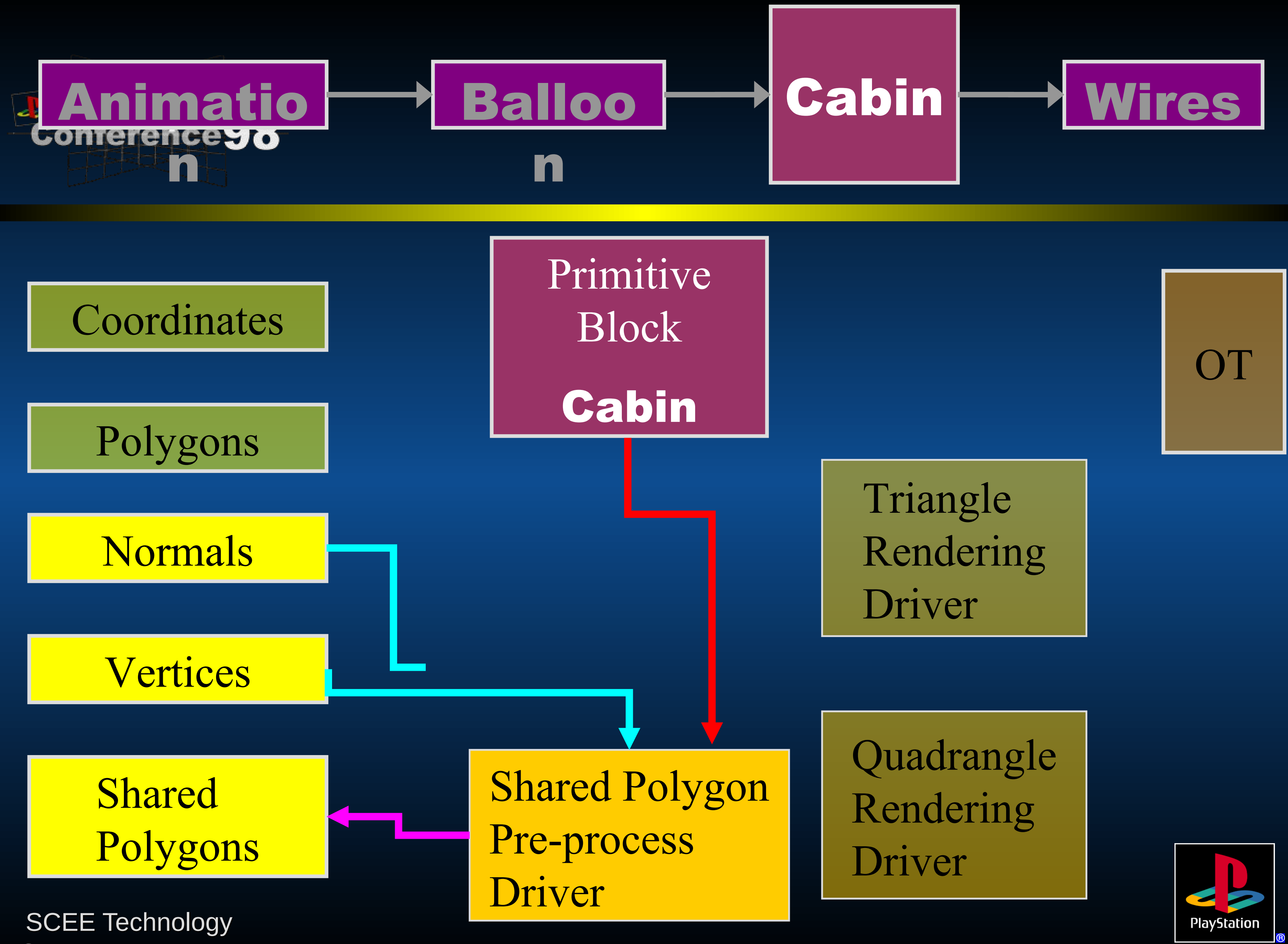

Sequence parameters
Interpolation parameters
Coordinates
Primitive Block
Animation
Frame Update Driver
Interpolation Drivers


Animatio
n
Balloo
n
Cabin
Wires
Coordinates
Polygons
Normals
Vertices
Shared Polygons
Primitive Block
Balloon
Triangle Rendering Driver
Quadrangle Rendering Driver
OT

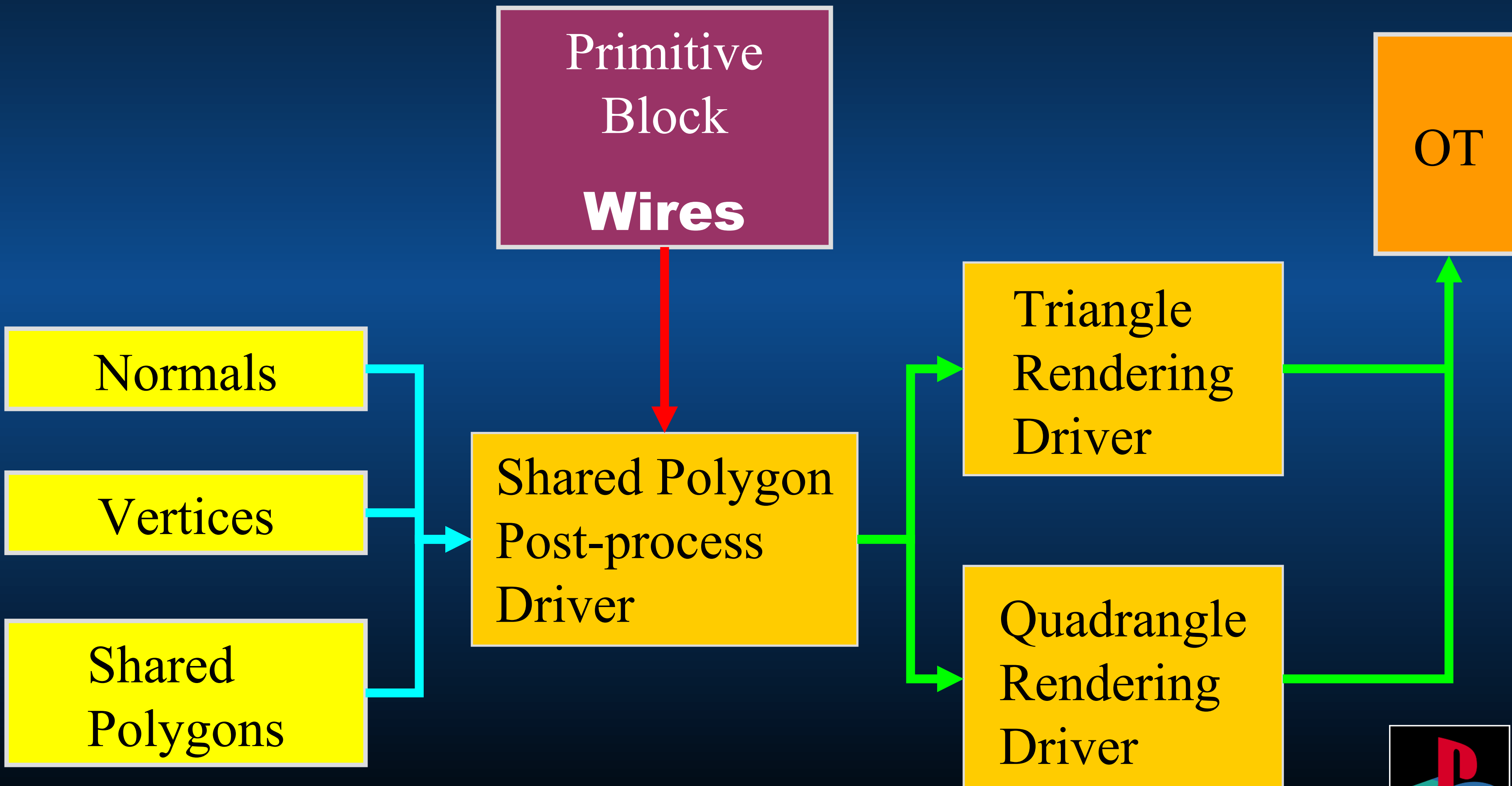

Animatio
n
Balloo
n
Cabin
Wires
Coordinates
Polygons
Normals
Vertices
Shared Polygons
Primitive Block
Balloon
Shared Polygon Pre-process Driver
Triangle Rendering Driver
Quadrangle Rendering Driver
OT


Animatio
n
Balloo
n
Cabin
Wires


Coordinates
Polygons
Normals
Vertices
Shared Polygons
Primitive Block
Cabin
Triangle Rendering Driver
Quadrangle Rendering Driver
OT



Animation
Balloon
Cabin
Wires
Conference98
Primitive Block
Cabin
OT
Coordinates
Polygons
Normals
Vertices
Shared Polygons
Triangle Rendering Driver
Shared Polygon Pre-process Driver
Quadrangle Rendering Driver
PlayStation


Primitive Block
Wires
OT
Normals
Vertices
Shared Polygons
Shared Polygon Post-process Driver
Triangle Rendering Driver
Quadrangle Rendering Driver

What did we just do?

- ANIMATION:
  - Worked out the changes to coordinate systems
- BALLOON & CABIN:
  - Sent GPU primitives to OT
  - Performed shared polygon pre-process
- WIRES:
  - Created shared polygons and...
  - …sent their primitives to the GPU

Summary

- Raw data is chunked into large sections.
- Primitives bind parts of that raw data together.
- Small and simple drivers operate on those primitives.
- Together, the Drivers create the final result.
- *Life is made easier for the programmer! (and the artists, and the producer…)*